

Policy-Based Reinforcement Learning

Optimal control with the dynamics replaced by data

Jinkyoo Park · KAIST

Lecture 9 *solved* for the feedback law. This lecture *learns* the same law.

— HANDOFF

The handoff — delete the dynamics

Lecture 9 owned the dynamics. This lecture takes them away and keeps the controller.

Where we are — the control lineage, made data-driven

STAGES
dynamic
○ — ●

MODEL · CROSSING
model-based → data-driven
○ — ●

AGENTS
single agent
● — ○

	LINEAGE A · OR / DYNAMIC PROGRAMMING	LINEAGE B · CONTROL THEORY
MODEL-BASED (ORIGIN)	MDP & Dynamic Programming LECTURE 7	Optimal Control / Planning LECTURE 9
DATA-DRIVEN (EXTENSION)	Value-Based RL LECTURE 8	Policy-Based RL LECTURE 10

We stay in the same cell of the big map — [single agent, multiple stages](#) — and move along **one axis only**: from a system whose f we are *handed* to one we can only *sample*. This is the move Lecture 8 made on the other column, performed now on ours. [The 2×2 closes here.](#)

What Lecture 9 gave us, stated as one object — the [feedback law](#):

$$u = \gamma(t, x), \quad \text{and in the one closed form,} \quad u = -Kx, \quad K = R^{-1}B^\top P$$

And three ways to obtain it — HJB, Riccati, Pontryagin. All three are exact. All three take f ([or \$A, B\$](#)) [as an input](#), and we no longer have it.

What we keep, and what we lose

The feedback law survives. **The dynamics do not.**

Everything else follows from removing f . Three things break the moment it vanishes:

- **The solvers.** HJB is a PDE *in* f ; the Riccati equation is built from A and B ; Pontryagin needs $\partial_x f$ to run the costate backwards. None of the three can even be written down.
- **The pointwise minimisation.** Every one of them ends in $\min_{u \in U} \{ g + \partial_x V \cdot f \}$ — a search over a continuum that needs f to evaluate. Lecture 8 met the same object as $\max_{a'} Q(s', a')$ and could not do it either.
- **The rollout.** We cannot integrate $\dot{x} = f$ to see what a candidate controller *would* do. We can only run it and watch.

So stop trying to compute the best action. **If you cannot search for the best action, learn to output it.**

The translation table — optimal control, learned

Every object of Lecture 9 has a data-driven counterpart. Keep this in sight all lecture.

LECTURE 9 (OPTIMAL CONTROL)	LECTURE 10 (POLICY-BASED RL)	WHAT REPLACES THE MODEL
value field $V(t, x)$, the HJB equation	critic V_w, Q_w	a <i>learned</i> value
feedback law $u = \gamma(t, x), u = -Kx$	actor $\pi_\theta(a \mid s) / \mu_\theta(s)$	a <i>learned</i> controller
known dynamics $\dot{x} = f(x, u)$	sampled transitions (s, a, r, s')	the environment itself
Pontryagin's costate, trajectory optimisation	REINFORCE, trajectory sampling	the score-function trick
solve Riccati / HJB	gradient ascent on $J(\theta)$	stochastic optimisation
controllability of (A, B)	sufficient exploration	a reachability assumption, either way

Read column 3. Nothing here is a new *principle* — it is Lecture 9's programme with the one piece we no longer own quietly swapped out. **Policy gradient is optimal control with f deleted and replaced by data.** DDPG's actor $\mu_\theta(s)$ is the learned cousin of LQR's gain K ; REINFORCE is Pontryagin's trajectory view with the dynamics sampled rather than known.

One honest detour — why not identify the system?

The most literal idea, and the exact rhyme of the detour Lecture 8 took: estimate the dynamics from data, then run Lecture 9 unchanged.

$$(\hat{A}, \hat{B}) = \arg \min_{A, B} \sum_t \|x_{t+1} - Ax_t - Bu_t\|^2 \implies \text{solve the Riccati equation for } \hat{K}$$

This is [system identification](#), it is the oldest idea in adaptive control, and it is perfectly valid.

But notice the same waste Lecture 8 noticed. To act we only ever need the controller — $m \times n$ numbers, or one network — yet here we first estimate a *whole dynamics model*, and then we act as though it were true. Why estimate f at all,

when we could estimate [the controller directly](#)?

That question — *skip the model, learn the thing that acts* — is the entire policy-based programme, and it is Lecture 8's question with Q^* replaced by π_θ . (The model comes back, deliberately and learned, in Lecture 11.)

The roadmap — four questions

One question per Act. This strip returns at every transition — watch the highlight move.



- **Q1 — How do we get a gradient without f ?** The **score-function trick**: the dynamics enter the trajectory's law and leave its log-derivative. (*Williams, 1992; Sutton et al., 2000*)
- **Q2 — Why is the estimate so noisy?** Causality, baselines, the advantage — and the **actor-critic**. (*Mnih et al., 2016*)
- **Q3 — How do we handle continuous control?** **DPG / DDPG**: the actor climbs $\nabla_a Q$ instead of searching it. (*Silver et al., 2014; Lillicrap et al., 2015*)
- **Q4 — How large a step dare we take?** Trust regions in policy space: **TRPO** and **PPO**. (*Schulman et al., 2015; 2017*)

— ACT 1

A gradient without the model

Q1. The thing we must differentiate is an expectation over trajectories, and the dynamics are inside it.

The objective, and a gradient we cannot see

Q1 Gradient?

Q2 Variance?

Q3 Continuous?

Q4 Safe step?

Parameterise a stochastic policy $\pi_\theta(a \mid s)$ and maximise the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)], \quad \theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

The trouble is where the parameter lives. It is not in the integrand; it is in the **distribution being integrated against**, and that distribution contains the dynamics:

$$p_\theta(\tau) = \mu(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

THE NAIVE ANSWER, AND WHY IT IS NOT ENOUGH

the source deck's own first attempt

Perturb one coordinate at a time and difference: $\partial J / \partial \theta_k \approx (J(\theta + \epsilon u_k) - J(\theta)) / \epsilon$. It works even when J is not differentiable — and it costs **one full batch of rollouts per parameter**. For a network with 10^6 weights that is not slow, it is impossible.

The score-function trick — the dynamics vanish

One identity does all the work: $\nabla p = p \nabla \log p$. It turns the gradient of an expectation into an expectation of a gradient:

$$\nabla_{\theta} \mathbb{E}_{\tau \sim p_{\theta}}[R(\tau)] = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau = \mathbb{E}_{\tau \sim p_{\theta}}[\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$$

Now expand that log. The product becomes a sum, and every term without θ differentiates to zero:

$$\nabla_{\theta} \log p_{\theta}(\tau) = \underbrace{\nabla_{\theta} \log \mu(s_0)}_{=0} + \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) + \underbrace{\sum_t \nabla_{\theta} \log P(s_{t+1} | s_t, a_t)}_{=0} = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

The start distribution and the dynamics contribute **exactly zero** to the gradient.

This is the precise sense in which f is "deleted": we never needed it. Where Lecture 9 *solved* the dynamics, here they simply drop out of the algebra — the policy's own log-probability is all that is left. We still *live* in the dynamics; we just never differentiate through them.

Reading the estimator — it is weighted maximum likelihood

Put the two halves together and the whole of Act 1 is one line:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\underbrace{\left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right)}_{\nabla_{\theta} J_{\text{ML}}(\theta) - \text{make this trajectory likelier}} \underbrace{\left(\sum_t r_t \right)}_{\text{how good it was}} \right]$$

The first factor, alone, is the gradient of the log-likelihood of the trajectory — [the maximum-likelihood gradient of Lecture 2](#), the direction that would make *this* trajectory more probable. The return simply weights it.

- **Increase** the likelihood of trajectories with large accumulated reward;
- **decrease** the likelihood of trajectories with small accumulated reward.

Policy gradient is [imitation of your own better moments](#), reweighted by how well they went.

There is no `arg max` anywhere in that sentence, and no model. It is also, exactly, Lecture 6's move — a generative model trained to emit the *good* samples — one axis over.

The policy gradient theorem, and REINFORCE

The dynamic-programming route (Backup 1) reaches the same place through Q^π rather than through trajectories, and gives the canonical statement:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a) \right]$$

REINFORCE makes it runnable by replacing the unknown Q^π with the return actually observed, since $\mathbb{E}_\pi[G_t \mid s, a] = Q^\pi(s, a)$: (Williams, 1992)

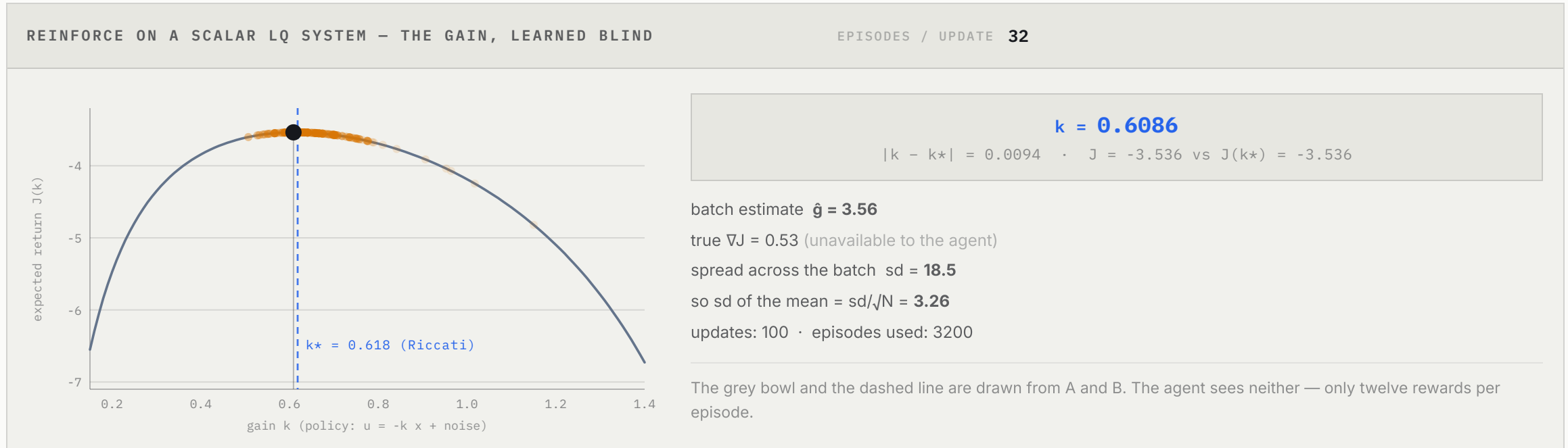
$$\theta \leftarrow \theta + \alpha \sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t$$

Written per action, per episode, or averaged over a batch of N episodes — the source deck writes all three. The γ^t is usually dropped in practice, and we drop it from here on.

It is **unbiased** — the estimator's mean *is* the gradient. And it is Monte Carlo, so we must wait for the episode to end, and every one of the T terms is multiplied by the *same* noisy tail. Unbiased and **badly high variance**. Taming that is Act 2.

REINFORCE, run — and what it converges to

a scalar linear system, learned blind



$x_{t+1} = x_t + u_t$ with cost $\sum (x^2 + u^2)$ — Lecture 9's problem, scalar. The policy is $u = -kx + \sigma\epsilon$ and the only parameter is the gain k . REINFORCE never sees A or B ; it only samples. The grey bowl $J(k)$ and the dashed line $k^* = 0.618$, the **Riccati gain**, are drawn for us , not for the agent — and that is where the gain walks to. Read the two numbers on the right: the estimate scatters *around* the true gradient, never away from it, and its spread falls like $1/\sqrt{N}$. Then set the batch to 8 and watch the gain stop settling — **that residual wander is Act 2's subject.**

— ACT 2

Taming variance with a critic

Q2. The estimator is unbiased. It is also nearly unusable. Three fixes, none of which adds bias.

Why the estimate is so noisy



Look again at what one episode contributes:

$$\hat{g}^{(i)} = \left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right)$$

Every score term is multiplied by the **same** scalar — the whole episode's return. One unlucky tail rescales all T of them together. And the return itself is a sum of T random rewards along a random trajectory.

Worse, it is *credit assignment by superstition*: an action at $t = 1$ is rewarded for something that happened at $t = 40$ and could not possibly have caused.

Three corrections follow, and **every one of them leaves the mean untouched**.

That constraint is the whole game. Anything that changes the mean changes what we are optimising; only variance is free to attack. (*"no bias, high variance — reduce the variance while keeping the bias unchanged", the source deck's own framing*)

Fix 1 — causality: half the terms were pure noise

An action at time t cannot affect a reward at time $t' < t$. So those cross-terms have mean zero and contribute **nothing but variance**. Drop them:

$$\nabla_{\theta} J = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t' \geq t} r_{t'} \right] = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{Q}_t \right]$$

The inner sum is the **reward-to-go** $\hat{Q}_t$ — an unbiased sample of $Q^{\pi}(s_t, a_t)$, which is why the trajectory derivation and the DP derivation meet here. The cost of this fix is one index. It is the cheapest variance reduction in reinforcement learning, and in the numbers of the Act 1 widget it removes about half.

Fix 2 — a baseline: subtract anything action-independent

Subtract from each reward-to-go a quantity $b(s_t)$ that does not depend on the action taken:

$$\nabla_{\theta} J = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (\hat{Q}_t - b(s_t)) \right]$$

THE BASELINE IS FREE

it adds *exactly zero* bias

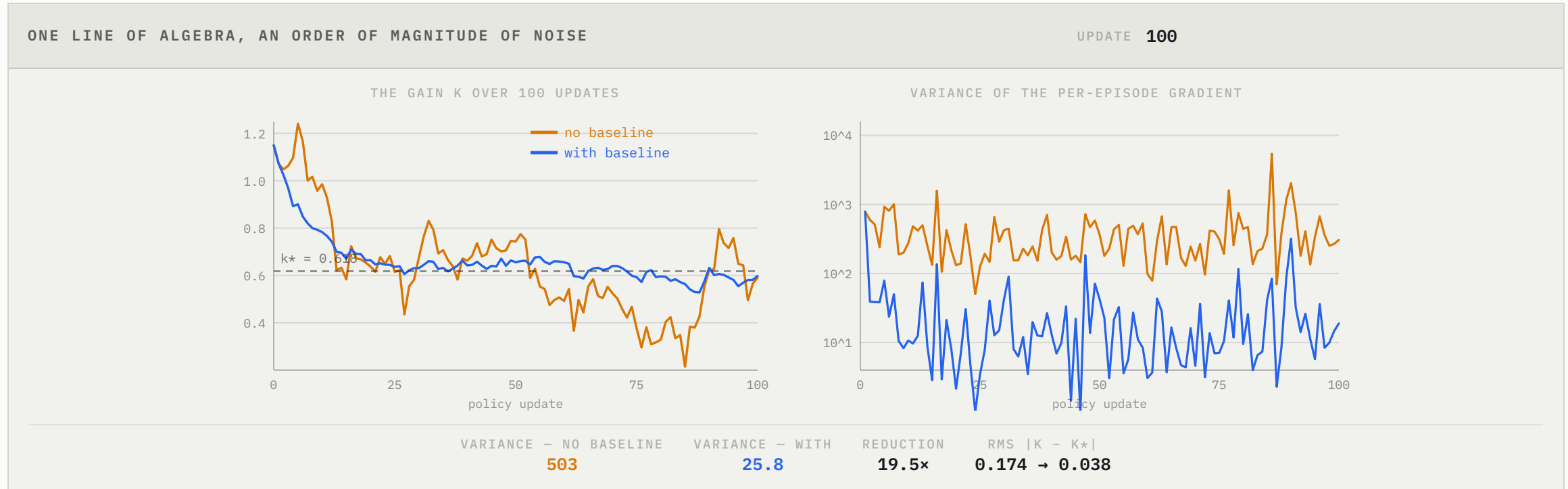
$$\mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) b(s)] = b(s) \int \frac{\nabla_{\theta} \pi_{\theta}(a | s)}{\pi_{\theta}(a | s)} \pi_{\theta}(a | s) da = b(s) \nabla_{\theta} \int \pi_{\theta}(a | s) da = b(s) \nabla_{\theta} 1 = 0$$

So *any* $b(s)$ is admissible, and we may choose the one that minimises variance. Doing that honestly (Backup 3) leaves a least-squares problem, $\min_b \mathbb{E}[(\hat{Q}_t - b(s_t))^2]$, whose solution is

$$b^*(s_t) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\hat{Q}_t] = V^{\pi}(s_t)$$

The critic is not a design choice. It is **the least-squares answer** to a variance question.

The baseline, measured



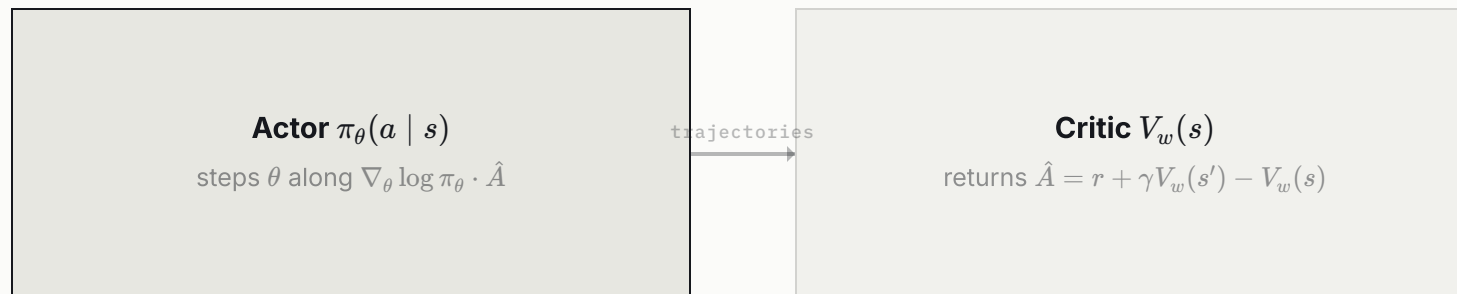
The same task, the same seed, the same starting gain, ten episodes per update — the only difference is whether $b(s_t)$ is subtracted. Left: the gain over 100 updates. Right: the variance of the per-episode gradient, on a log axis. The baseline changes **the mean of the estimator not at all** and its spread by nearly **twenty-fold**, which is the difference between a run that settles on k^* and one that keeps wandering past it.

Fix 3 — the advantage, and the actor–critic

Put the two together. With $b(s) = V^\pi(s)$ and $\hat{Q}_t \rightarrow Q^\pi(s_t, a_t)$, what multiplies the score is the **advantage**:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad \text{— “how much better than average was this action?”}$$

And now we need a *learned* V . Learn both:



That $\hat{A}$ is a **TD error** — Lecture 8's engine, running inside a policy update. Here the two lineages briefly touch: a control-lineage actor, steadied by an OR-lineage critic. The three families of the field are not three; they are two circles and their intersection, and the intersection is where almost everything that works actually lives.

Two dials on the same idea

what the field does with the actor–critic once it has one

- **A3C / A2C** — run many actors in parallel on separate copies of the environment. At any instant they occupy different states, so their data is decorrelated *without* a replay buffer: **parallelism plays the role of replay**, and lets the method stay on-policy. (*Mnih et al., 2016*)
- **n -step returns** — bootstrap after n steps rather than at the end: $R_t = \sum_{i=1}^n \gamma^{i-1} r_{t+i-1} + \gamma^n V(s_{t+n})$. REINFORCE is $n = T$, one-step actor–critic is $n = 1$.
- **GAE** — do not pick n ; average all of them geometrically, $\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}^V$. (*Schulman et al., 2016*)

GAE is **TD(λ) for the advantage** — Lecture 8's bias–variance dial, moved from the value to the thing that trains the policy.

— ACT 3

Continuous control: DDPG

Q3. Lecture 8 handed over a wall. This is where it falls.

The wall Lecture 8 left



The source deck states the debt in one sentence: *"although DQN solves problems with high-dimensional observation spaces, it can only handle discrete and low-dimensional action spaces"*. There are exactly two ways to put a continuous action into a Q-network, and both break.

OPTION 1 – DISCRETISE THE ACTION

One output head per action, as in DQN. Take $-1 \leq a \leq 1$ and cut it into bins of width Δa .

What is a good Δa ? Coarse bins mean poor control resolution; fine bins mean n^d heads for d joints. And the optimum is **never exactly on the grid**.

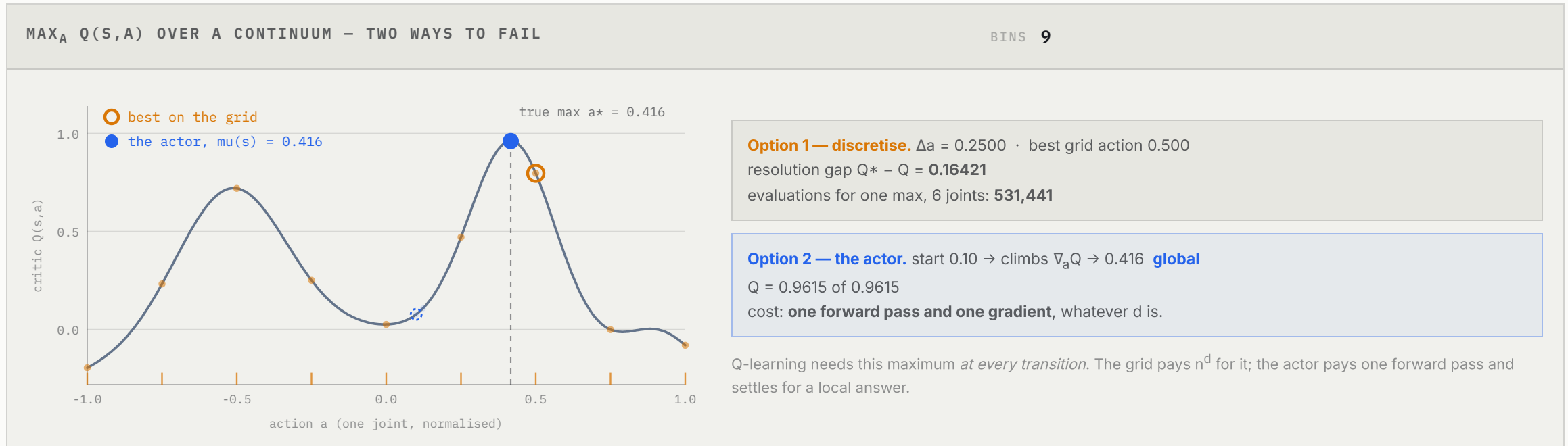
OPTION 2 – FEED THE ACTION IN

One network $Q_w(s, a)$ with the action as an input, so the action stays continuous.

But the update needs $\max_{a'} Q_w(s', a')$, and Q_w is **non-convex in a'** . That is a global optimisation, at every transition, forever.

Option 1 keeps the update and breaks the action space; option 2 keeps the action space and breaks the update. **There is no third way that keeps Q-learning**, which is why the answer has to come from the other lineage.

The wall, measured



One state, one continuous action, and the critic's $Q(s, \cdot)$ across it. Nine bins already cost $9^6 = 531,441$ evaluations per transition on a six-joint arm — and still miss the peak by 0.16. Twenty-one bins close the gap and cost **85,766,121**. The actor emits its action in **one forward pass** and refines it by following $\nabla_a Q$: cheap, and only ever *local* — move its start and watch it settle on the wrong hill, which is honest, and is why TD3 and SAC exist.

The deterministic fix — the actor climbs the critic

Make the actor **deterministic**, $a = \mu_\theta(s)$, and let it ascend the critic's action-gradient: (*Silver et al., 2014*)

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s \sim \rho^\mu} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q(s, a) \Big|_{a=\mu_\theta(s)} \right]$$

This is how the intractable $\max_a Q$ is finally beaten: **the actor follows $\nabla_a Q$ uphill instead of searching for its summit.**

Determinism is not a stylistic preference. Compare the Bellman expectation in the two cases:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r, s' \sim E} \left[r + \gamma \mathbb{E}_{a' \sim \pi} [Q^\pi(s', a')] \right] \quad \text{versus} \quad Q^\mu(s_t, a_t) = \mathbb{E}_{r, s' \sim E} \left[r + \gamma Q^\mu(s', \mu(s')) \right]$$

With a deterministic μ the **inner expectation disappears** and only the environment is left inside the expectation. Two consequences, both decisive: the critic can be trained **off-policy** on any data at all, and there is one integral to estimate instead of two — **less variance**.

DDPG — deterministic policy gradient, plus every DQN stabiliser

Lillicrap et al., 2015 — DQN (2013) → DPG (2014) → DDPG (2016)

THE TWO LOSSES

Critic, by Bellman error on a replayed minibatch:

$$L(w) = \frac{1}{N} \sum_i \left(Q_w(s_i, a_i) - [r_i + \gamma Q_{w^-}(s_{i+1}, \mu_{\theta^-}(s_{i+1}))] \right)^2$$

Actor, by the deterministic policy gradient:

$$\nabla_{\theta} J \approx \frac{1}{N} \sum_i \nabla_a Q_w(s, a) \big|_{a=\mu_{\theta}(s_i)} \nabla_{\theta} \mu_{\theta}(s_i)$$

THE STABILISERS, RETURNING

- **replay buffer** — off-policy is now permitted, so use it;
- **target networks** w^-, θ^- , updated *softly* every step, $w^- \leftarrow \tau w + (1 - \tau)w^-$, rather than copied every C steps;
- both are Lecture 8's deadly-triad fixes, unchanged. The paper's own verdict on ablating them: **"target networks matter a lot"**.

And nobody implements the DPG theorem. In code the actor loss is written `pi_loss = -Q(s, mu(s))` and `.backward()` is called; autograd's chain rule $\partial_{\theta} Q(s, \mu_{\theta}(s)) = \partial_a Q \cdot \partial_{\theta} \mu_{\theta}$ **is** the theorem. A page of mathematics collapses into one line because the chain rule was the whole content.

A deterministic actor explores nothing

A deterministic policy has no randomness to explore with, and the obvious repair — act uniformly at random — is a poor one. The source deck gives three reasons, and the third is unusual in this course:

- random actions drive the policy update into regions where the critic **is not accurate** (Lecture 5's adversarial-optimiser hazard, in an RL costume);
- exploring *unseen states* needs action that is **consistent along the episode**, not resampled each step;
- and on real hardware, **random jittering breaks the machine**.

THE PENDULUM ARGUMENT

the source deck's own picture

Swinging a pole upright takes a *sustained* push in one direction, then alternation near the top. White noise averages to nothing over an episode and never sustains anything, so a uniformly random policy might eventually find the swing-up — after enormously many samples.

Hence the **Ornstein–Uhlenbeck** process, $dx_t = -\kappa x_t dt + \sigma dW_t$, added to $\mu_\theta(s)$: noise that is *temporally correlated*, so exploration pushes rather than jitters.

$\mu_\theta(s)$ is the learned K — the lineage, made literal

Put Lecture 9's Act 3 and this act side by side. They are the same object, twice.

	LQR (LECTURE 9)	DDPG (HERE)
the controller	$u = -Kx$	$a = \mu_\theta(s)$
how it is obtained	solve the Riccati equation	ascend $\nabla_a Q$ from samples
does it need the model?	yes — A, B, Q, R	no — sampled transitions
what it is	an $m \times n$ matrix	a neural network
optimality	global, exactly	local, approximately

Same feedback law, two ways to find it: [solve the dynamics, or sample their gradient](#).

Lecture 9 closed by asking you to hold $u = -Kx$ in view as "the closed form that policy gradient learns to approximate blind". That is this table, and the Act 1 widget is that sentence run as an experiment: a gain that walks to the Riccati answer without ever meeting A or B .

— ACT 4

Stepping without falling

Q4. We have a gradient and we have tamed its variance. How far along it dare we step?

The step-size trap unique to RL



In supervised learning a bad step costs one bad update; the next minibatch arrives from the same fixed dataset and pulls you back. In RL it is worse, because

the policy generates its own next batch of data.

A step that pushes π_θ too far lands in a region where the policy is bad — and now *every* subsequent sample comes from that bad policy. The gradient estimated there points somewhere else again. The damage is **self-reinforcing**, and there is no fixed dataset to fall back on.

This is also why the parameter step size is the wrong thing to control. A tiny change in θ can produce a large change in *behaviour* (a softmax near saturation), and a large one can produce none at all. What must be bounded is the distance between the old and new **action distributions**.

A trust region, in policy space

Maximise an importance-weighted advantage, subject to staying inside a **KL ball** around the policy that collected the data: *(Schulman et al., 2015)*

$$\max_{\theta} \mathbb{E} \left[\frac{\pi_{\theta}(a | s)}{\pi_{\theta_{\text{old}}}(a | s)} \hat{A} \right] \quad \text{s.t.} \quad \mathbb{E} [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | s) || \pi_{\theta}(\cdot | s))] \leq \delta \quad \Rightarrow \quad \text{monotonic improvement}$$

The ratio is Act 2's off-policy importance weight, reused: the data is from $\pi_{\theta_{\text{old}}}$, the objective is about π_{θ} . Solved by linearising the objective and quadratising the KL through the Fisher matrix — a natural-gradient step (Backup 5). Powerful, and heavy.

THIS IS LECTURE 1'S TRUST REGION, MOVED

the **trust-region** widget of Chapter 1, in policy space

LECTURE 1 • SUCCESSIVE CONVEXIFICATION

TRPO

a quadratic model of f

a linear model of the advantage

trust radius ρ in parameter space

KL radius δ in **behaviour** space

accept if the model was believable, else shrink

backtrack until the true KL and improvement hold

Same object, same logic: **optimise a local model only as far as you are entitled to believe it**. Go back and run that widget; it is the same accept-or-shrink test, with a divergence in place of a distance.

PPO — the clip that does the same job for nothing

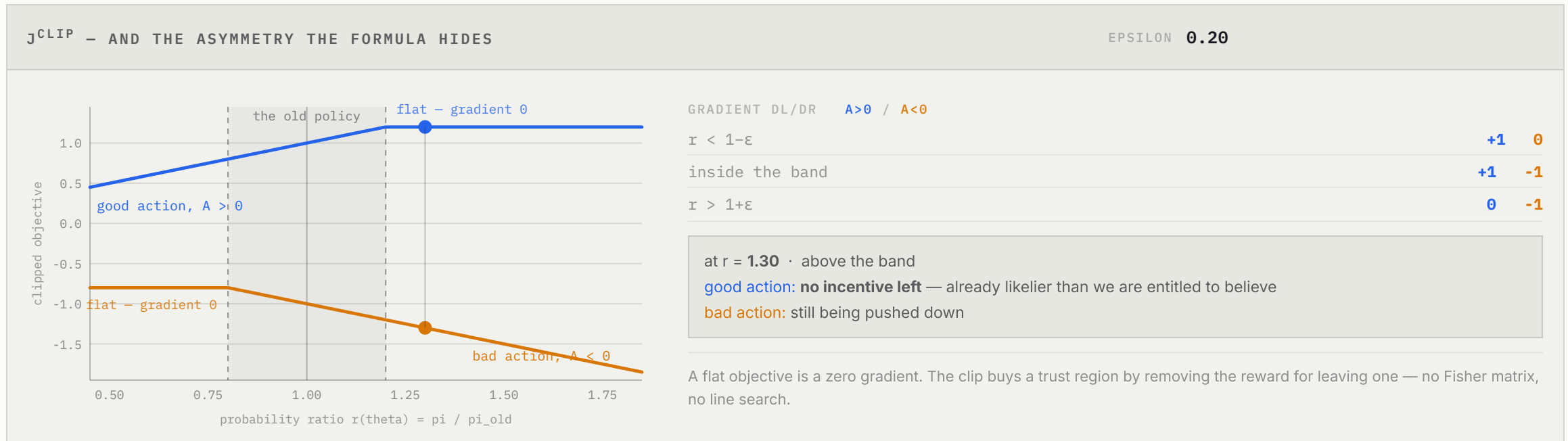
TRPO's constraint is exact and expensive. PPO keeps the intent and throws away the machinery: with $r(\theta) = \pi_\theta(a \mid s) / \pi_{\theta_{\text{old}}}(a \mid s)$, (Schulman et al., 2017)

$$J^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r(\theta) \hat{A}, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A} \right) \right]$$

No second-order machinery, no conjugate gradient, no line search. When the ratio leaves $[1 - \epsilon, 1 + \epsilon]$ *in the direction the update was pushing it*, the objective goes flat and **the incentive to push further is simply removed**.

With shared actor–critic parameters the practical objective adds two terms — $-c_1(V_\theta(s) - V_{\text{target}})^2$ for the critic and $+c_2 H(\pi_\theta(\cdot \mid s))$, an entropy bonus for exploration. The entropy term is the seed of maximum-entropy RL and of SAC.

The clip, and the asymmetry nobody mentions



The clipped objective as a function of the ratio, for a good action ($\hat{A} > 0$) and a bad one ($\hat{A} < 0$). Read the slopes: for $\hat{A} > 0$ the gradient is **exactly zero above $1 + \epsilon$** — no reward for making a good action still likelier. For $\hat{A} < 0$ it is zero *below* $1 - \epsilon$ but **stays alive above $1 + \epsilon$** : an action already too probable and known to be bad keeps being pushed down. The clip only removes the incentive that would take you out of the region.

— CLOSING

Closing

Both lineages have now had the model taken away. The grid is full.

Where we are — both lineages, now data-driven

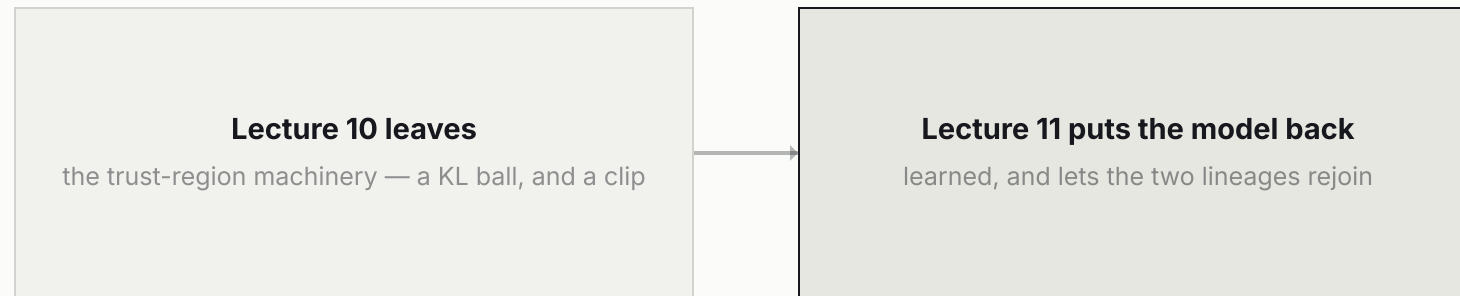
	LINEAGE A • OR / DYNAMIC PROGRAMMING	LINEAGE B • CONTROL THEORY
MODEL-BASED (ORIGIN)	MDP & Dynamic Programming LECTURE 7	Optimal Control / Planning LECTURE 9
DATA-DRIVEN (EXTENSION)	Value-Based RL LECTURE 8	Policy-Based RL LECTURE 10

Two traditions, each with a model-based origin and a data-driven extension, and the same move relates each pair:

	MODEL-BASED ORIGIN	WHAT IS DELETED	DATA-DRIVEN EXTENSION
Lineage A • OR	MDP & DP (<i>Lec 7</i>)	the model (P, R)	value-based RL (<i>Lec 8</i>)
Lineage B • Control	optimal control (<i>Lec 9</i>)	the dynamics f	policy-based RL (<i>Lec 10</i>)

One orphaning move, made twice: **delete the model, sample instead.**

What we hand on



The trust region travels further than it looks. It is Lecture 1's ratio test in behaviour space; it is what makes PPO the default workhorse of modern policy-based RL; and it is the discipline every method needs the moment **its own output decides what it will see next** — which is exactly the condition Lecture 12 will face with no interaction at all.

Lecture 11 asks the question both extensions have been avoiding: if deleting the model cost us this much, what happens if we *learn* it — and plan with it, and let an optimal-control teacher train a policy student?

Lecture 9 found the feedback law by solving the dynamics. Lecture 10 finds *the same law* by sampling its gradient.

The dynamics never solved, only experienced — and the controller learned, not derived.

Questions?

Read against Lecture 8's closing and the symmetry is exact: value-based RL kept the Bellman equation and threw away the model; policy-based RL kept the feedback law and threw away the dynamics. Two parents, one orphaning move. And the through-line of every algorithm today is a single expression, $\mathbb{E}[\nabla_{\theta} \log \pi_{\theta} \cdot \hat{A}]$ — they differ only in *which advantage they trust* and *how large a step they dare*.

— APPENDIX

Backup slides

Complete arguments, kept out of the narrative.

Backup 1 — the policy gradient theorem, the DP route (i): a recursion

The trajectory derivation of Act 1 is one of two. Here is the other, which starts from the value function and never leaves the Bellman equation. Take $J(\theta) = \sum_s d^\pi(s) V^\pi(s)$ with d^π the on-policy stationary distribution, and differentiate $V^\pi(s) = \sum_a \pi_\theta(a | s) Q^\pi(s, a)$ by the product rule:

$$\nabla_\theta V^\pi(s) = \sum_a [\nabla_\theta \pi_\theta(a | s) Q^\pi(s, a) + \pi_\theta(a | s) \nabla_\theta Q^\pi(s, a)]$$

Expand $Q^\pi(s, a) = \sum_{s'} P(s' | s, a)[r + V^\pi(s')]$. Since P carries no θ the gradient passes straight through it — **the same cancellation as Act 1, in a different costume** — and only $V^\pi(s')$ survives. With $\phi(s) = \sum_a \nabla_\theta \pi_\theta(a | s) Q^\pi(s, a)$ and $\rho^\pi(s \rightarrow s', 1) = \sum_a \pi_\theta(a | s) P(s' | s, a)$ that is a recursion:

$$\nabla_\theta V^\pi(s) = \phi(s) + \sum_{s'} \rho^\pi(s \rightarrow s', 1) \nabla_\theta V^\pi(s')$$

Backup 2 — the policy gradient theorem, the DP route (ii): unrolled

Substitute the recursion into itself. Composing one-step visitations gives k -step ones, and the whole tail collects into a single sum:

$$\nabla_{\theta} V^{\pi}(s) = \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \rho^{\pi}(s \rightarrow x, k) \phi(x)$$

Set $\eta(s) = \sum_k \rho^{\pi}(s_0 \rightarrow s, k)$ and normalise it into a probability distribution. The constant $\sum_s \eta(s)$ is absorbed into the step size, which is why the theorem is usually stated with α :

$$\nabla_{\theta} J(\theta) \propto \sum_s d^{\pi}(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) Q^{\pi}(s, a) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi}(s, a)] \quad \blacksquare$$

The two routes differ in what they make visible. The DP route shows that the result is a Bellman object and connects it to Lecture 8; [the trajectory route shows the transition kernel entering and leaving](#), which is the claim this chapter is built on.

Backup 3 — the baseline: unbiased, and the best one

Unbiasedness, in full. Split the trajectory expectation at time t ; everything after t integrates out and the score has mean zero under π_θ :

$$\mathbb{E}_\tau [\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = \mathbb{E}_{s_{0:t}, a_{0:t-1}} \left[b(s_t) \cdot \mathbb{E}_{a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t)] \right] = \mathbb{E}[b(s_t) \cdot 0] = 0$$

$$\text{since } \mathbb{E}_{a_t} [\nabla_\theta \log \pi_\theta(a_t | s_t)] = \int \frac{\nabla_\theta \pi_\theta(a_t | s_t)}{\pi_\theta(a_t | s_t)} \pi_\theta(a_t | s_t) da_t = \nabla_\theta \int \pi_\theta(a_t | s_t) da_t = \nabla_\theta 1 = 0$$

Which baseline. Treat the per-step terms as approximately independent and the score and the return as approximately uncorrelated:

$$\text{Var} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) (\hat{Q}_t - b(s_t)) \right] \approx \sum_t \mathbb{E} [\|\nabla_\theta \log \pi_\theta(a_t | s_t)\|^2] \mathbb{E} [(\hat{Q}_t - b(s_t))^2]$$

The first factor does not involve b . So minimising the variance is exactly $\min_{b(s_t)} \mathbb{E}[(\hat{Q}_t - b(s_t))^2]$ — least squares — with solution $b^*(s_t) = \mathbb{E}[\hat{Q}_t] = V^\pi(s_t)$.

Two consequences worth stating. The baseline must be **re-fitted as the policy moves**, because V^π moves with π ; and a baseline that depends on the *action* would not be free — the whole argument above rests on b passing outside the expectation over a_t . (Action-dependent baselines can be made unbiased with extra correction terms; that is a research literature, not a free lunch.)

Backup 4 — GAE, and the λ dial

With $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$, the k -step advantage estimators telescope:

$$\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k})$$

$k = 1$ is the one-step TD advantage: low variance, biased by whatever error V carries. $k = \infty$ is the Monte-Carlo advantage: unbiased, and as noisy as REINFORCE. GAE takes the exponentially weighted average of all of them, (*Schulman et al., 2016*)

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} := (1 - \lambda) (\hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \cdots) = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V$$

which collapses to a single running sum of TD errors. $\lambda = 0$ recovers one-step actor-critic, $\lambda = 1$ recovers Monte-Carlo.

This is **TD(λ) applied to the advantage** rather than to the value, and the discount γ is doing double duty: as the problem's discount, and as a variance-reduction parameter that downweights delayed effects at the cost of bias. Lecture 8's Backup 1 drew the same dial for MC-versus-TD; it is the same dial.

Backup 5 — TRPO's natural-gradient step, in closed form

Maximise $L(\theta) = \mathbb{E}[r(\theta)\hat{A}]$ subject to $\bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \leq \delta$.

Step 1 — local models. To first and second order about θ_{old} ,

$$L(\theta) \approx g^\top (\theta - \theta_{\text{old}}), \quad \bar{D}_{\text{KL}} \approx \frac{1}{2} (\theta - \theta_{\text{old}})^\top H (\theta - \theta_{\text{old}})$$

with $g = \nabla_\theta L$ and H the Fisher information matrix. The KL has zero gradient at θ_{old} , which is why its leading term is quadratic.

Step 2 — a linear objective on an ellipsoid. The maximiser lies along the natural-gradient direction $H^{-1}g$, scaled to the KL radius:

$$\theta_{\text{new}} = \theta_{\text{old}} + \sqrt{\frac{2\delta}{g^\top H^{-1}g}} H^{-1}g$$

Step 3 — practice. $H^{-1}g$ is obtained by conjugate gradient without ever forming H , and the step is backtracked until the *true* KL and the *true* improvement both hold — which is Lecture 1's accept-or-shrink test, verbatim.

PPO discards all of it and recovers the effect with one clip on $r(\theta)$. That is the usual pattern in this course: an exact method that characterises the answer, and a cheap one that reaches it — HJB and LQR, TRPO and PPO.

Backup 6 — the policy-based zoo, placed

All of them are $\mathbb{E}[\nabla_{\theta} \log \pi_{\theta} \hat{A}]$, or its deterministic form $\mathbb{E}[\nabla_{\theta} \mu_{\theta} \nabla_a Q]$. They differ only in **which advantage they trust** (Act 2) and **how large a step they dare** (Act 4).

METHOD	KEY IDEA	REFERENCE
REINFORCE	Monte-Carlo policy gradient, no critic	Williams, 1992
A3C / A2C	parallel actor–critic; parallelism replaces replay	Mnih et al., 2016
GAE	λ -weighted advantage	Schulman et al., 2016
DPG / DDPG	deterministic actor climbs $\nabla_a Q$; off-policy	Silver 2014; Lillicrap 2015
TD3	twin critics, delayed actor — the overestimation fix	Fujimoto et al., 2018
TRPO	KL trust region, monotonic improvement	Schulman et al., 2015
PPO	clipped surrogate; the workhorse	Schulman et al., 2017
SAC	maximum-entropy off-policy actor–critic	Haarnoja et al., 2018